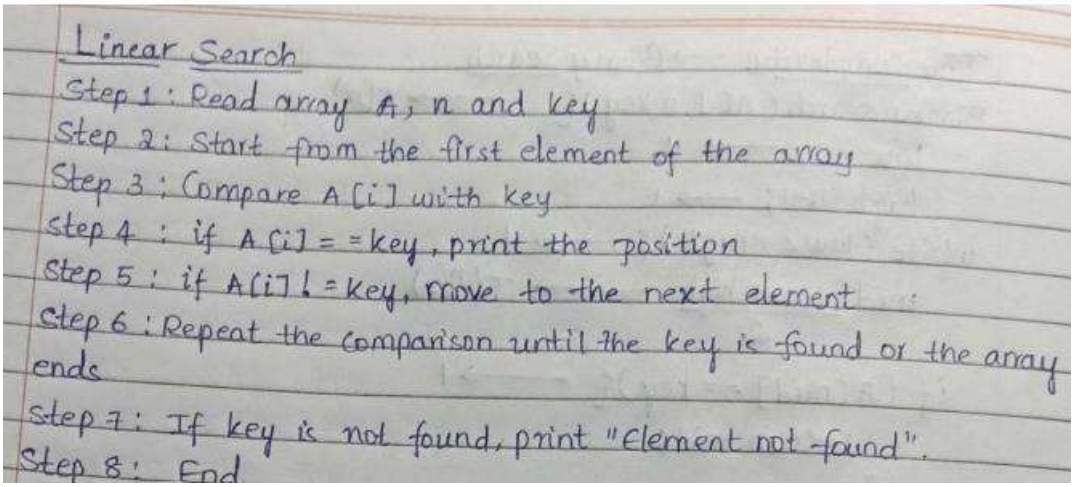
# Practical-2

**Aim:** To Implementation and Time analysis of linear and binary search algorithm.

## 1)Linear search

## Algorithm:



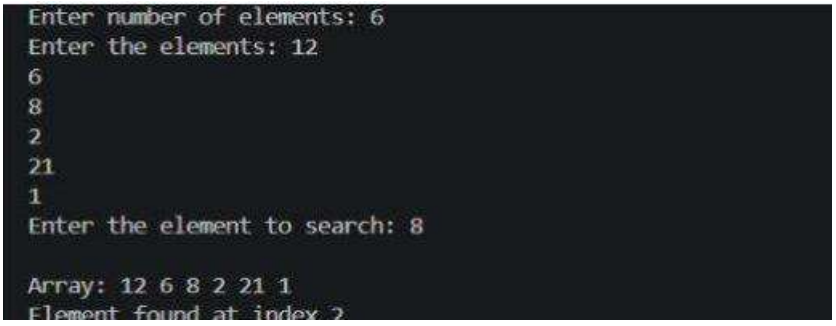

## Program:

```
#include <iostream>

using namespace std;

class LinearSearch

{

public:

   int arr[100], n, key;

   void arrayInput()

   {

      cout << "Enter number of elements: ";

      cin >> n;

      cout << "Enter the elements: ";
```

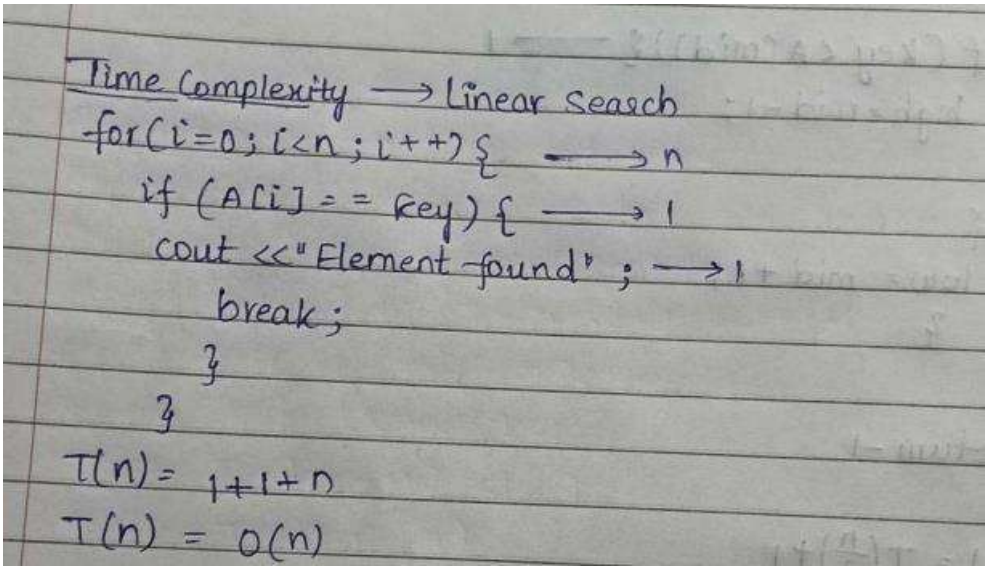
```
    for (int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }
    cout << "Enter the element to search: ";
    cin >> key;
}
void printArray()
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}
void search()
{
    int i;
    for (i = 0; i < n; i++)
    {
        if (arr[i] == key)
        {
            cout << "Element found at index " << i << endl;
            break;
        }
```

```
        }
        if (i == n)
        {
            cout << "Element not found." << endl;
        }
    }
};
int main()
{
    LinearSearch ls;
    ls.arrayInput();
    cout << "\nArray: ";
    ls.printArray();
    ls.search();
    return 0;
}
```

## Output:

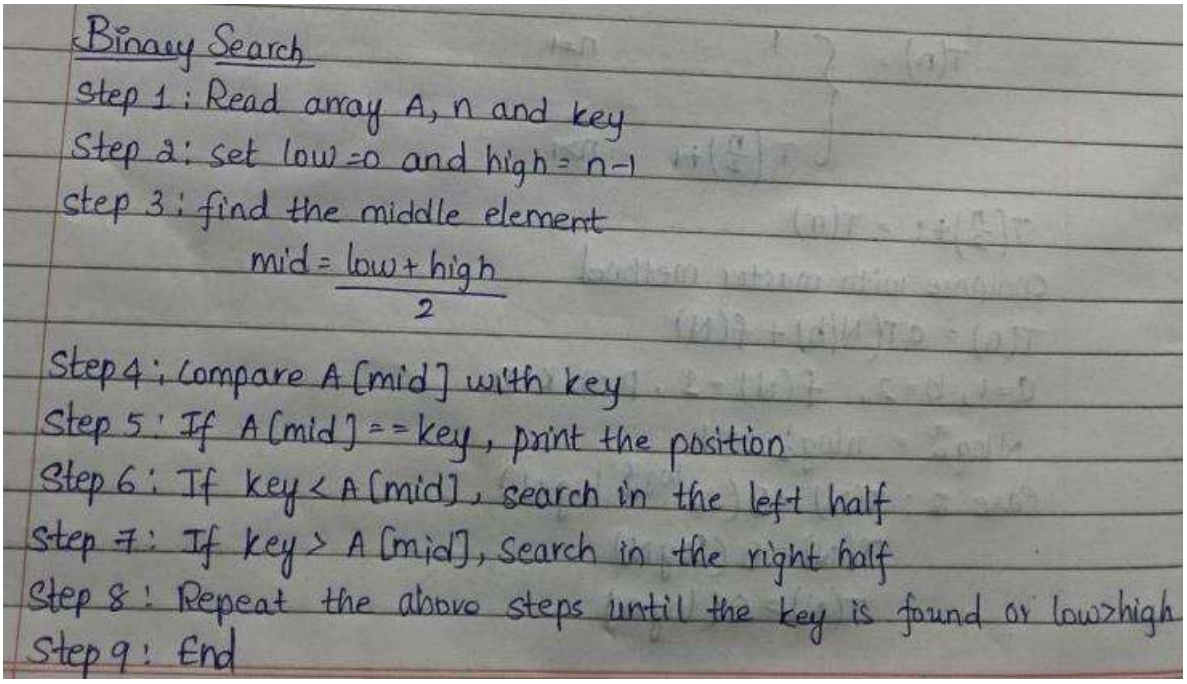


**Time complexity:**




**2)Binary search**

**Algorithm:**




Name:V.Bhavya sri
Enrollment no: 92460118227

## Program:

```
#include <iostream>
using namespace std;
class BinarySearch
{
public:
    int arr[100], n, key;
    void arrayInput()
    {
        cout << "Enter number of elements: ";
        cin >> n;
        cout << "Enter the sorted elements: ";
        for (int i = 0; i < n; i++)
        {
            cin >> arr[i];
        }
        cout << "Enter the element to search: ";
        cin >> key;
    }
    void printArray()
    {
        for (int i = 0; i < n; i++)
        {
            cout << arr[i] << " ";
        }
```

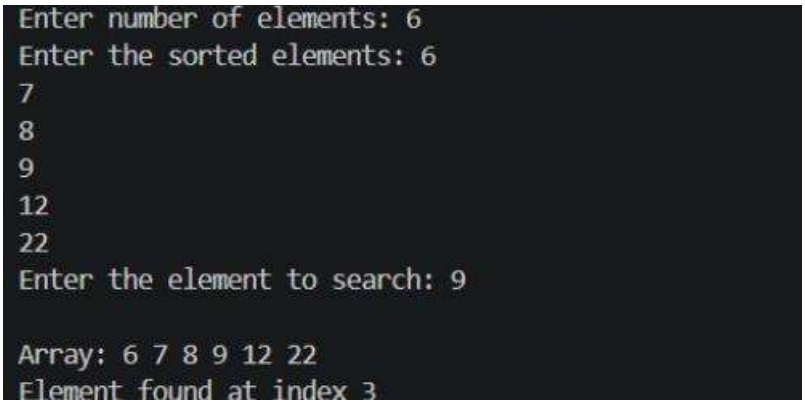
Name:V.Bhavya sri
Enrollment no: 92460118227

```
    cout << endl;
}
void search()
{
    int low = 0;
    int high = n - 1;
    while (low <= high)
    {
        int mid = (low + high) / 2;
        if (arr[mid] == key)
        {
            cout << "Element found at index " << mid << endl;
            return;
        }
        else if (key < arr[mid])
        {
            high = mid - 1;
        }
        else
        {
            low = mid + 1;
        }
    }
    cout << "Element not found." << endl;
}
```

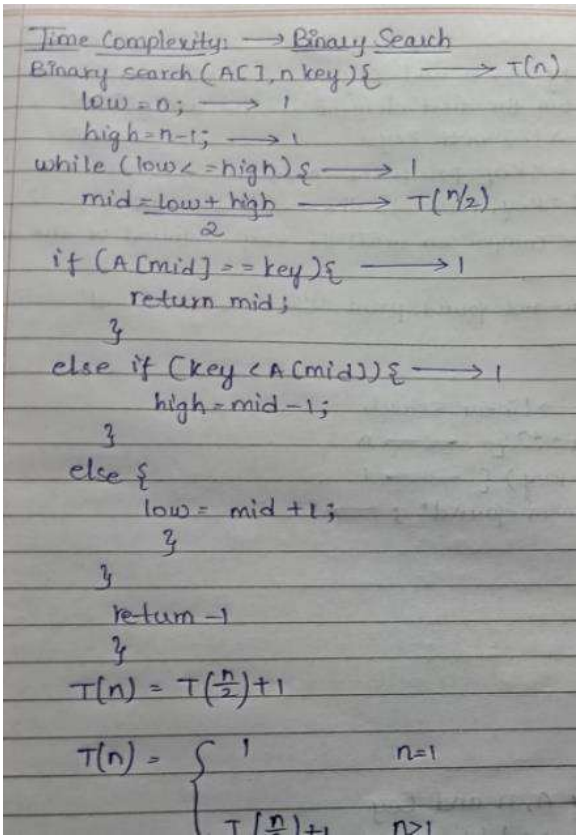
Name:V.Bhavya sri
Enrollment no: 92460118227

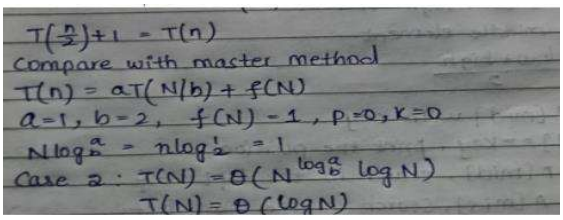
```
};
int main()
{
    BinarySearch bs;
    bs.arrayInput();
    cout << "\nArray: ";
    bs.printArray();
    bs.search();
    return 0;
}
```

## Output:



## Time complexity:




## Conclusion :

The implementation and time analysis of Linear Search and Binary Search helps us understand their efficiency. Linear Search is simple and can work on any array, but it may require **O(n)** comparisons. Binary Search is faster with **O(log n)** time in average and worst cases, but it requires the array to be sorted. Therefore, Binary Search is more efficient for searching large sorted datasets.

Name:V.Bhavya sri
Enrollment no: 92460118227